Studio Nx

AI 구현 설명 83 — Web Audio API 쉽게 이해하기

2026-10-19

83 — Web Audio API 쉽게 이해하기

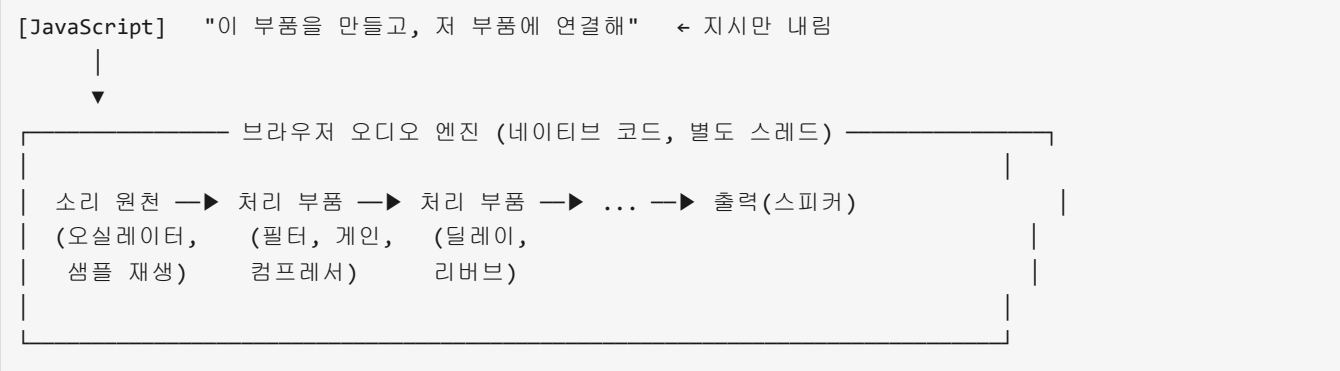
항목	내용
주제	이 프로젝트의 소리가 나는 원리인 Web Audio API (브라우저에 내장된 오디오 기능)
대상	Web Audio API를 처음 접하는 사람, 프로젝트 질문에 답해야 하는 사용자
작성	Claude (AI), 2026-10-19, 사용자 검토
함께 볼 문서	개요 82, Default_Instruments_Effects.md

---

1. 한 줄 요약

Web Audio API는 브라우저가 기본으로 제공하는 오디오 처리 부품 모음입니다. JavaScript는 부품을 고르고 전선처럼 연결하는 지시만 내리고, 실제 소리 계산은 브라우저 안의 별도 오디오 엔진이 맡습니다. 그래서 설치할 것이 없고, JavaScript가 바빠도 소리가 끊기지 않습니다.

2. 큰 그림



- 비유하면 모듈러 신스나 믹싱 콘솔의 패치 케이블과 같습니다. 부품(노드)을 꺼내서 케이블(연결)로 이으면 소리가 흐릅니다.
- JavaScript는 연결 구조를 만들 뿐 샘플 하나하나를 계산하지 않습니다. 계산은 엔진이 합니다.

3. 꼭 알아야 할 용어 6가지

용어	뜻	이 프로젝트의 예
`AudioContext`	오디오 세계 전체입니다. 시계와 출력 장치를 가지고 있고, 모든 노드는 여기서 만들어집니다.	`AudioEngine`이 하나를 만듭니다 (`audio/engine.js`)
노드 (`AudioNode`)	소리를 만들거나 가공하는 부품 하나입니다.	`GainNode`, `BiquadFilterNode` 등
연결 (`connect`)	한 노드의 출력을 다음 노드의 입력에 잇는 일입니다.	`source.connect(gain); gain.connect(destination);`
`destination`	맨 끝의 출력입니다. 여기에 이어진 소리가 스피커로 나갑니다.	믹서의 마지막 노드가 연결됩니다

용어	뜻	이 프로젝트의 예
`AudioParam`	노드의 조절 값입니다. 값을 바로 바꾸거나, 정해진 시각에 바뀌도록 예약할 수 있습니다.	`gain.gain`, `filter.frequency`
`currentTime`	엔진의 시계(초)입니다. 모든 예약은 이 시계를 기준으로 합니다.	스케줄러의 `clock.now()`

4. 가장 작은 예제

440Hz 소리를 1초 동안 내는 전체 코드입니다.

```
const ctx = new AudioContext();            // 오디오 세계를 만듭니다
const osc = ctx.createOscillator();        // 소리 원천: 사인파 발생기
const gain = ctx.createGain();             // 처리 부품: 볼륨
osc.frequency.value = 440;                 // 라(A4)
gain.gain.value = 0.2;                     // 볼륨 20%

osc.connect(gain);                         // 오실레이터 → 게인
gain.connect(ctx.destination);             // 게인 → 스피커

osc.start(ctx.currentTime);                // 지금 시작
osc.stop(ctx.currentTime + 1);             // 1초 뒤 정지
```

이 세 줄(`create`, `connect`, `start`)의 반복이 Web Audio의 전부입니다. 이 프로젝트의 Synth, 이펙트, 믹서도 같은 일을 더 많이 하는 것뿐입니다.

5. 부품 종류와 이 프로젝트에서 쓰는 곳

5.1 소리 원천

노드	하는 일	쓰는 곳
`OscillatorNode`	사인, 톱니, 사각, 삼각파를 만듭니다.	Synth의 오실레이터와 LFO, 메트로놈 클릭
`AudioBufferSourceNode`	디코딩한 샘플(`AudioBuffer`)을 재생합니다. 재생 속도로 음높이를 바꿀 수 있습니다.	오디오 클립 재생 (`audio/playback.js`)

5.2 처리 부품

노드	하는 일	쓰는 곳
`GainNode`	소리 크기를 곱합니다.	트랙 볼륨, 엔벨로프, 이펙트의 믹스 양
`StereoPannerNode`	좌우 위치를 정합니다.	트랙 팬, Gain 이펙트의 팬
`BiquadFilterNode`	로우패스, 하이패스, 선반, 피킹 등 필터입니다.	Synth 필터, EQ 3-Band, Delay의 톤
`DelayNode`	소리를 정해진 시간만큼 늦춥니다.	Delay 이펙트, Reverb의 프리딜레이
`ConvolverNode`	잔향 소리(임펄스 응답)를 곱해서 공간감을 만듭니다.	Reverb
`DynamicsCompressorNode`	큰 소리를 눌러 줍니다.	Compressor, Limiter
`ChannelSplitterNode`	좌우 채널을 따로 꺼냅니다.	미터

5.3 엿보기와 출력

노드	하는 일	쓰는 곳
`AnalyserNode`	지나가는 소리를 바꾸지 않고 파형만 읽습니다.	레벨 미터, 헤드리스 시험의 피크 측정
`destination`	스피커 출력입니다.	마스터의 마지막 노드

6. 시간 다루기: 가장 중요한 부분

음악은 박자가 정확해야 합니다. 여기서 흔한 실수와 이 프로젝트의 해법을 설명합니다.

6.1 `setTimeout`으로 박자를 맞추면 안 되는 이유

`setTimeout`이나 `setInterval`은 브라우저가 바쁘면 수십 밀리초씩 늦어집니다. 박자가 흔들리는 소리가 납니다. 반대로 오디오 엔진의 시계(`currentTime`)는 매우 정확합니다.

6.2 해법: 미리 예약하기 (lookahead scheduler)

```
JavaScript 타이머 (25ms마다)     "앞으로 0.1초 안에 울릴 노트를 지금 예약해 둬"
          |
          ▼
오디오 엔진 시계                  예약된 시각에 정확히 울림 (타이머가 늦어도 상관없음)
```

- 타이머는 대략적으로만 돌아도 됩니다. 소리 시각은 엔진 시계로 정하기 때문입니다.
- 이 프로젝트의 `Transport`(`audio/scheduler.js`)가 이 방식입니다. 25ms마다 깨어나서 0.1초 앞까지의 노트를 `start(when)`으로 예약합니다.

6.3 값도 예약할 수 있습니다

`AudioParam`은 시각을 정해서 값을 바꿀 수 있습니다. Synth의 엔벨로프(ADSR)가 이 기능입니다.

메서드	뜻
`setValueAtTime(값, 시각)`	그 시각에 값을 즉시 바꿉니다.
`linearRampToValueAtTime(값, 시각)`	그 시각까지 직선으로 값을 바꿉니다.
`setTargetAtTime(값, 시각, 시간상수)`	목표 값에 지수적으로 다가갑니다. 부드럽게 바꿀 때 씁니다.

```
// 어택 0.01초, 디케이 0.3초, 서스테인 0.7의 엔벨로프 (audio/synth.js의 playSynthNote 요약)
amp.gain.setValueAtTime(0, when);
amp.gain.linearRampToValueAtTime(peak, when + 0.01);
amp.gain.linearRampToValueAtTime(peak * 0.7, when + 0.31);
```

볼륨 노브를 돌릴 때 값을 바로 바꾸면 "틱" 하는 잡음이 납니다. 그래서 믹서는 `setTargetAtTime`으로 아주 짧게 이어서 바꿉니다.

7. 브라우저의 자동 재생 제한

브라우저는 사용자가 클릭하기 전에는 소리를 내지 못하게 막습니다. `AudioContext`가 `suspended` 상태로 시작하기 때문에, 재생 버튼을 누르는 순간 `ctx.resume()`을 불러야 합니다. `AudioEngine.start()`가 이 일을 합니다.

8. 파일로 만들기: `OfflineAudioContext`

`OfflineAudioContext`는 스피커 없이 소리를 계산만 하는 오디오 세계입니다. 재생 시간만큼 기다리지 않고 최대한 빠르게 계산해서 결과를 배열로 줍니다.

```
const ctx = new OfflineAudioContext(2, 44100 * 10, 44100);   // 2채널, 10초, 44.1kHz
// ... 실시간과 같은 방식으로 노드를 만들고 연결 ...
const buffer = await ctx.startRendering();                    // 10초를 몇 초 만에 계산
// buffer 를 WAV 헤더와 함께 저장하면 WAV 파일이 됩니다 (S1-11b)
```

이 프로젝트의 핵심 설계가 여기에 있습니다. 믹서, Synth, 이펙트가 모두 "컨텍스트를 인자로 받는" 코드라서, 같은 코드가 실시간 재생과 WAV 출력 양쪽에 쓰입니다. 그래서 출력 파일의 소리가 화면에서 듣던 소리와 같아집니다. 이펙트 시험(`effects_offline.mjs`)도 이 방식으로 실제 소리를 계산해서 확인합니다.

9. 한계와 대안

한계	설명	대안
기본 부품만으로는 정밀한 DSP가 어렵습니다	정확한 리미터, 고급 리버브, 보코더, 피치 시프터는 기본 노드로 만들 수 없습니다.	`AudioWorklet`에 직접 만든 처리 코드를 올립니다.
직접 만든 처리는 JavaScript가 느립니다	빠른 계산이 필요한 처리는 C++이 유리합니다.	C++ (JUCE)로 만들어 WebAssembly(WASM)로 `AudioWorklet`에서 실행합니다.
브라우저마다 약간 다릅니다	같은 노드도 구현 세부(예: 컴프레서의 자동 보정)가 다를 수 있습니다.	이 프로젝트는 Chrome과 Edge를 기준으로 시험합니다.
노드 연결 수가 많으면 부담이 됩니다	노트마다 오실레이터를 만드는 Synth는 동시 발음이 많으면 무거워집니다.	동시 발음 수 제한을 둘 수 있습니다 (지금은 제한이 없음).

계획 문서의 `engine='WEBAUDIO'` 구분이 이 한계를 반영합니다. 지금은 Web Audio 기본 부품으로 만든 이펙트로 시작하고, 사용자의 C++ 학습 트랙(C-02 이후)에서 만든 WASM 이펙트를 `engine='WASM'`으로 나중에 교체합니다.

10. 이 프로젝트에서 실제로 부딪힌 세 가지

이 문제들은 문서나 시험 없이는 알기 어려워서 기록해 둡니다. 모두 이펙트 시험이 실패해서 발견했습니다.

문제	증상	원인	해결
모노 소리가 가운데에서 작아집니다	Gain 이펙트를 -12dB로 했더니 -15dB로 들렸습니다.	`StereoPannerNode`는 모노 입력을 가운데에 놓을 때 각 쪽에 0.707배(-3dB)를 줍니다(등출력 팬 법칙).	입력을 먼저 스테레오로 늘려서(`channelCountMode = 'explicit'`) 가운데에서는 크기가 그대로가 되게 했습니다(`createStereoGain`).
컴프레서가 스스로 소리를 키웁니다	천장 -12dB로 둔 Limiter가 입력보다 큰 소리를 냈습니다.	`DynamicsCompressorNode`는 압축으로 줄어든 만큼 자동으로 키웁니다 (0dB 입력에서의 게인의 -0.6 제곱).	그 값을 계산해서 같은 양만큼 줄였습니다(`compressorAutoMakeupDb`).
딜레이의 되먹임이 박자에서 늦어집니다	0.1초 간격으로 반복해야 하는데 두 번째 반복이 0.2초가 아니라 0.2029초에 울렸습니다.	되먹임 고리를 한 바퀴 돌 때마다 브라우저가 128샘플(약 2.9ms)을 더 걸리게 합니다.	고리의 딜레이 시간에서 128샘플을 빼고, 첫 반복은 따로 늦춰서 모든 반복이 정확한 간격이 되게 했습니다.

11. 자주 받을 질문

Q. Web Audio API는 JavaScript로 만든 것인가요? A. 아닙니다. 브라우저가 네이티브 코드로 구현해서 JavaScript에 노출한 기능입니다. 그래서 JavaScript가 느려도 오디오 계산은 빠릅니다.

Q. 따로 설치하거나 라이브러리를 받아야 하나요? A. 필요 없습니다. Chrome, Edge, Firefox, Safari에 기본으로 들어 있습니다. 이 프로젝트도 외부 오디오 라이브러리를 쓰지 않습니다.

Q. VST 같은 외부 플러그인을 쓸 수 있나요? A. 직접 쓸 수는 없습니다. 브라우저에서는 Web Audio의 부품과 `AudioWorklet`으로 만든 처리만 쓸 수 있습니다. 그래서 이 프로젝트는 신스와 이펙트를 직접 만듭니다.

Q. 소리가 안 나면 먼저 무엇을 확인하나요? A. 순서대로 확인합니다. (1) `ctx.state`가 `running`인지(자동 재생 제한), (2) 마지막 노드가 `destination`에 이어졌는지, (3) 트랙 볼륨, 뮤트, 솔로, (4) 이펙트의 값(컷오프 20Hz처럼 극단 값)을 봅니다. 이 프로젝트에서 실제로 컷오프 20Hz 때문에 소리가 거의 안 났던 적이 있습니다.

Q. 실시간 재생과 WAV 출력의 소리가 같다는 걸 어떻게 보장하나요? A. 같은 코드가 컨텍스트만 바꿔 쓰이기 때문입니다. 잔향의 임펄스 응답도 고정 시드 난수로 만들어서, 같은 설정이면 늘 같은 소리가 납니다.

Q. 이 문서의 내용은 어디서 확인하나요? A. 노드 사용은 `studionx/frontend/static/js/audio/` 폴더의 파일들입니다. 세 가지 문제의 시험은 `docs/handoff/s1-11a_tests/effects_offline.mjs`에서 재현됩니다.